# Lecture # 3

Multithreading /Cache coherence/Protocol/Synchronization

By: Muhammad Zain Uddin

email: zuddin@iba.edu.pk

# Multithreading

```
#include <stdio.h>
#include <pthread.h>
#include <stdlib.h>
typedef struct {
    int start;
    int end;
    int* data;
    int* partial_sum;
} thread_data_t;

void* worker(void* arg) {
    thread_data_t* data = (thread_data_t*)arg;
    int sum = 0;
    for (int i = data->start; i < data->end; ++i) {
        sum += data->data[i] * data->data[i];
    }
    int* result = (int*)malloc(sizeof(int));
    *result = sum;
    return result;
}
int sum_of_squares_multithread(int data[], int len, int num_threads)
{
    pthread_t threads[num_threads];
    thread_data_t thread_data[num_threads];
```

```
for (int i = 0; i < num_threads; ++i) {
        thread_data[i].start = i * (len / num_threads);
        thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
        thread_data[i].data = data;
        thread_data[i].partial_sum = NULL;
        pthread_create(&threads[i], NULL, worker, &thread_data[i]);
    }

    int total_sum = 0;
    for (int i = 0; i < num_threads; ++i) {
        void* result;
        pthread_join(threads[i], &result);
        total_sum += *(int*)result;
        free(result);
    }

    return total_sum;
}

int main() {
    int data[] = {2, 5, 7, 4};
    int sum = sum_of_squares_multithread(data, 4, 2);
    printf("Sum of squares: %d\n", sum);

    return 0;
}
```

**Headers:**
stdio.h: Provides functions for input/output operations like printing to the console.
pthread.h: Contains functions and types for working with threads, enabling multithreading.
stdlib.h: Offers general utility functions, including memory allocation and deallocation.

**Structure:**
thread_data_t: A structure for packaging information to be passed to worker threads:
Start: The starting index of the data segment assigned to the thread.
end: The ending index of the data segment.
data: A pointer to the shared array containing the data to be processed.

**Worker Function:**
worker:
Takes a thread_data_t pointer as input, providing thread-specific data.
Calculates the sum of squares within its assigned data segment.
Dynamically allocates memory using malloc to store the calculated partial sum.
Stores the partial sum in the allocated memory location.
Returns a pointer to the newly allocated memory containing the partial sum.

```
void* worker(void* arg) {
    thread_data_t* data = (thread_data_t*)arg;
    int sum = 0;
    for (int i = data->start; i < data->end; ++i) {
        sum += data->data[i] * data->data[i];
    }
    int* result = (int*)malloc(sizeof(int));
    *result = sum;
    return result;
}
```

**Sum of Squares Function:**

sum_of_squares_multithread:

Accepts the data array, its length, and the desired number of threads as input.

Creates an array of threads and thread_data_t structures to organize thread-related information.

Divide the data array into chunks and assign them to different threads for parallel processing.

Initializes a variable total_sum to 0, which will accumulate the final result.

Creates the threads using pthread_create, each executing the worker function with its corresponding thread_data_t.

Waits for all threads to finish using pthread_join, collecting their partial sums.

Iterates through the returned partial sums, adding them to total_sum.

Frees the memory allocated for partial sums using free to prevent memory leaks.

Returns the calculated total_sum, representing the sum of squares of all elements in the array.

**Main Function:**

main:

Creates a sample data array to be processed.

Calls the sum_of_squares_multithread function to perform the multithreaded calculation.

Prints the final sum of squares obtained from the multithreaded operation.

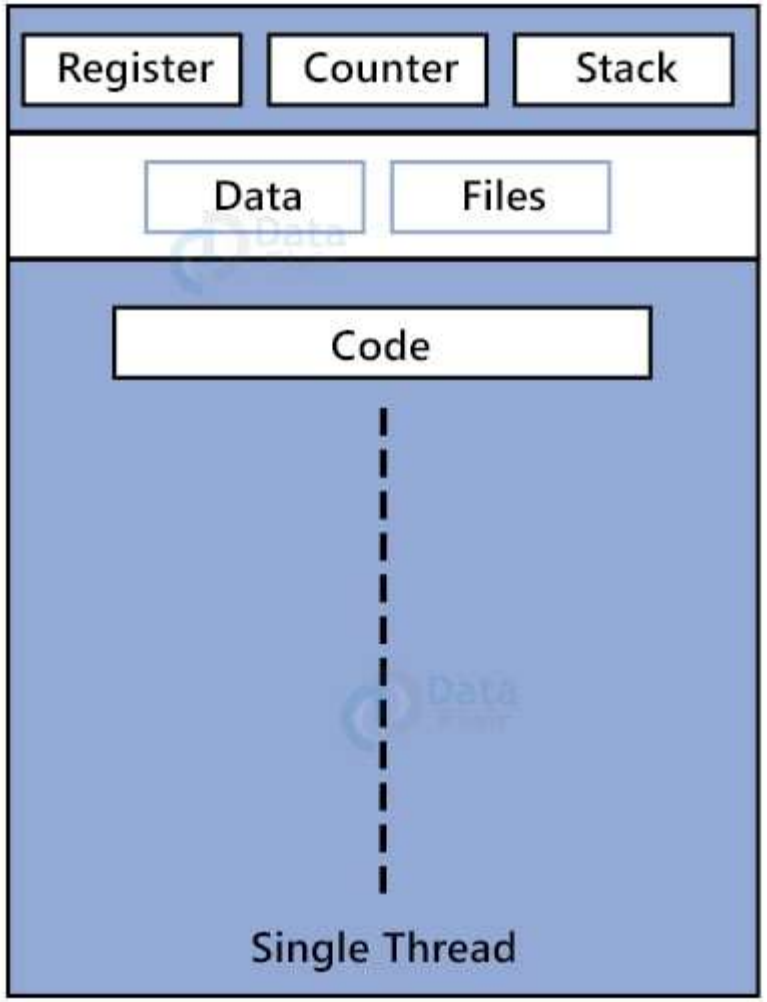
Returns 0 to indicate successful

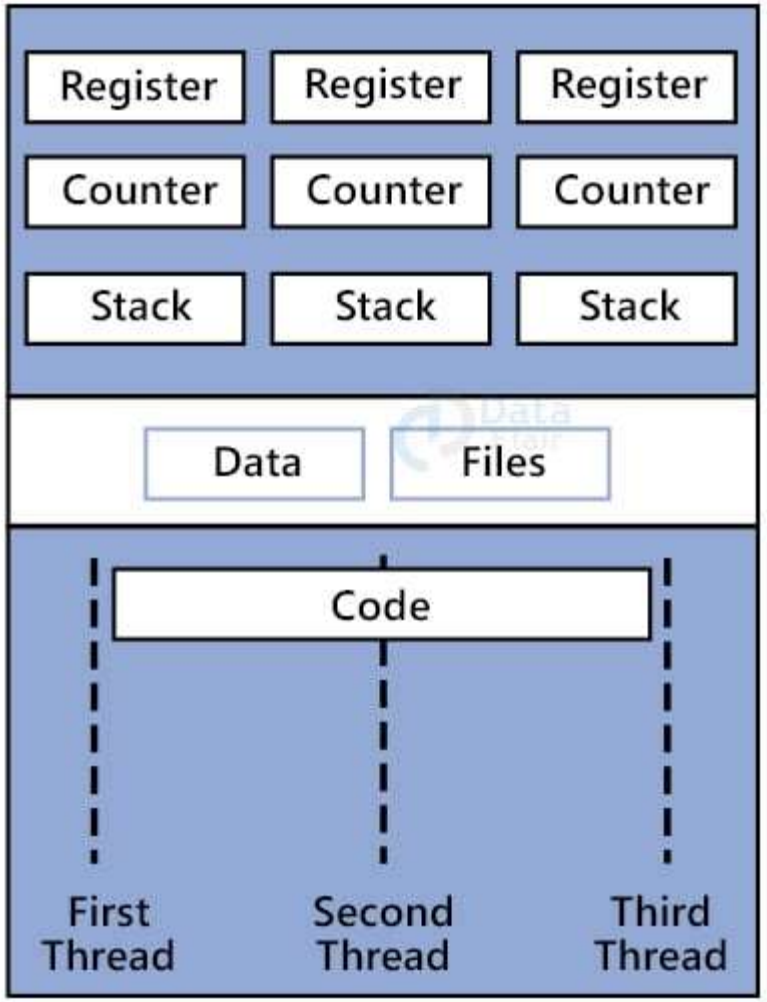
```
int sum_of_squares_multithread(int data[], int len, int num_threads) {
    pthread_t threads[num_threads];
    thread_data_t thread_data[num_threads];

for (int i = 0; i < num_threads; ++i) {
        thread_data[i].start = i * (len / num_threads);
        thread_data[i].end = (i == num_threads - 1) ? len : (i + 1) * (len / num_threads);
        thread_data[i].data = data;
        thread_data[i].partial_sum = NULL;
        pthread_create(&threads[i], NULL, worker, &thread_data[i]);
    }

    int total_sum = 0;
    for (int i = 0; i < num_threads; ++i) {
        void* result;
        pthread_join(threads[i], &result);
        total_sum += *(int*)result;
        free(result);
    }

    return total_sum;
}
```


Register
Counter
Stack
Data
Files
Code
Single Thread


Register
Register
Register
Counter
Counter
Counter
Stack
Stack
Stack
Data
Files
Code
First Thread
Second Thread
Third Thread

# Caching with multithreading

- Each core has its own L1 cache, and sometimes its own L2 cache
- Most caches are write-back, so it doesn't update main memory until the block gets evicted
- Problem for multithreaded code: If multiple threads are running with shared memory, how do we guarantee that the correct version of data is being used?
  - Can't access main memory anymore, if another cache has dirty data

# Caching Example: Multithreaded Interleaved Writes

- Imagine we run Matrix Multiply on 2 threads with interleaved writes
- The below shows one block in the L1 caches of each thread, and the right shows the state of the main memory

| Thread | V | Dirty | Data |
|---|---|---|---|
| 0 | 0 | 0 | 0x14 0xFD 0xBE 0x5E |

| Thread | V | Dirty | Data |
|---|---|---|---|
| 1 | 0 | 0 | 0xB5 0x81 0x67 0x3F |

Bt

| | | | |
|---|---|---|---|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

A

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

C

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

# Caching Example: Multithreaded Interleaved Writes

- Thread 0 computes C[0]
  - Since the cache is write-back, we don't write back to main memory immediately
  - For spatial locality, we copy the full block to the cache

| Thread | V | Dirty | Data |
|---|---|---|---|
| 0 | 1 | 1 | 0xFA 0x?? 0x?? 0x?? |

| Thread | V | Dirty | Data |
|---|---|---|---|
| 1 | 0 | 0 | 0xB5 0x81 0x67 0x3F |

Bt

| | | | |
|---|---|---|---|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

A

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 8 | 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

C

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

# Caching Example: Multithreaded Interleaved Writes

- Thread 1 computes C[1]
  - Main memory hasn't updated, so the cache block thread 1 gets doesn't have C[0] computed

| Thread | V | Dirty | Data |
|---|---|---|---|
| 0 | 1 | 1 | 0xFA 0x?? 0x?? 0x?? |

| Thread | V | Dirty | Data |
|---|---|---|---|
| 1 | 1 | 1 | 0x?? 0x04 0x?? 0x?? |

Bt

| | | | |
|---|---|---|---|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

A

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

C

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

# Caching Example: Multithreaded Interleaved Writes

- Thread 0 computes C[2]
- Thread 1 computes C[3]

| Thread | V | Dirty | Data |
|---|---|---|---|
| 0 | 1 | 1 | 0xFA 0x?? 0x0E 0x?? |

| Thread | V | Dirty | Data |
|---|---|---|---|
| 1 | 1 | 1 | 0x?? 0x04 0x?? 0x18 |

Bt

| | | | |
|---|---|---|---|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

A

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

C

| | | | |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |

# Caching Example: Multithreaded Interleaved Writes

- Thread 0 evicts the C[0] block

| Thread | V | Dirty | Data |
|---|---|---|---|
| 0 | 0 | 0 | 0xFA 0x?? 0x0E 0x?? |

| Thread | V | Dirty | Data |
|---|---|---|---|
| 1 | 1 | 1 | 0x?? 0x04 0x?? 0x18 |

Bt

| | | | |
|---|---|---|---|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

A

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 11 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

C

| | | | |
|---|---|---|---|
| FA | ?? | 0E | ?? |
| | | | |
| | | | |
| | | | |

# Caching Example: Multithreaded Interleaved Writes

- Thread 1 evicts the C[0] block
  - Overwrites the data from Thread 0!

| Thread | V | Dirty | Data |
|---|---|---|---|
| 0 | 0 | 0 | 0xFA 0x?? 0x0E 0x?? |

| Thread | V | Dirty | Data |
|---|---|---|---|
| 1 | 0 | 0 | 0x?? 0x04 0x?? 0x18 |

Bt

| | | | |
|---|---|---|---|
| 17 | 21 | 25 | 29 |
| 18 | 22 | 26 | 30 |
| 19 | 23 | 27 | 31 |
| 20 | 24 | 28 | 32 |

A

| | | | |
|---|---|---|---|
| 1 | 2 | 3 | 4 |
| 5 | 6 | 7 | 8 |
| 9 | 10 | 11 | 12 |
| 13 | 14 | 15 | 16 |

C

| | | | |
|---|---|---|---|
| ?? | 04 | ?? | 18 |
| | | | |
| | | | |
| | | | |

# Caching with multithreading

- Each core has its own L1 cache, and sometimes its own L2 cache
- Most caches are write-back, so it doesn't update main memory until the block gets evicted
- Problem for multithreaded code: If multiple threads are running with shared memory, how do we guarantee that the correct version of data is being used?
  - Can't access main memory anymore, if another cache has dirty data
- Three main cases
  - Two threads read
  - Two threads write
  - One thread reads, one thread writes

# Caching with multithreading

- Synchronous reads:
  - Should be allowed; as long as no one changes data, things are safe
- Synchronous writes:
  - Once two different threads have conflicting data, it's difficult to return to a reasonable state
  - Example: git merge conflicts
  - Therefore, we should not allow these to happen
  - If threads attempt to cause this, we need to force a thread to evict
  - These evictions cause **coherence misses**; in a multithreaded system with shared memory, coherence misses can cause significant thrashing
- Multiple threads read, one writes
  - Could go the safe route and force evictions
    - But causes coherence misses
  - Ideally: Allow threads to read the written data from another core's cache, or set up a way for one cache to update other caches
    - Needs some kind of communication protocol

# Introduction to Cache Coherence

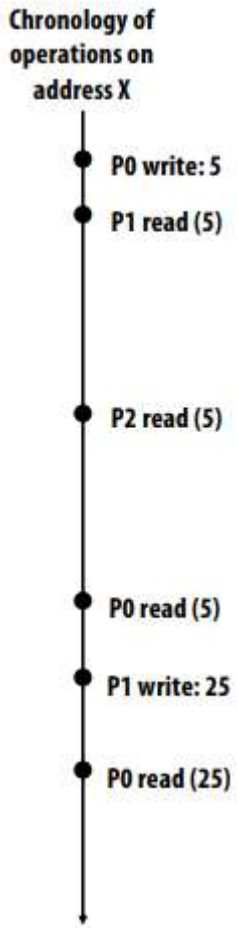
- **Definition:**
  - Cache coherence refers to the consistency of shared data in multiple caches that store copies of the same data.
- **Importance:**
  - Ensures that all processors in a multiprocessor system observe a consistent view of memory.

# Coherence Definition

A memory system is coherent if:

The results of a parallel program's execution are such that for each memory location, there is a hypothetical serial order of all program operations (executed by all processors) to the location that is consistent with the results of execution, and:

- Memory operations issued by any one processor occur in the order issued by the processor
- The value returned by a read is the value written by the last write to the location… as given by the serial order



# MOESI

- Solution: Set up a system where you can "snoop" on other caches to see if they have a cache block with the same tag
- Instead of just having a valid and dirty bit, have five different states for a block
  - Modified
  - Owned
  - Exclusive
  - Shared
  - Invalid
- Invalid: Same as valid bit off in regular cache; the block isn't in the cache
- Exclusive: Same as valid bit on, dirty bit off in regular cache
  - The block has been read, but not modified
  - Further, no other cache has this block
- Modified: Same as valid bit on, dirty bit on in regular cache
  - The block has been read, and modified
  - Further, no other cache has this block

# MOESI

- Shared
  - The block is in some other cache
  - We haven't modified this block, and we're not allowed to make modifications
    - If we need to make modifications, evict the block from everyone's cache and move to modified
  - Allows for synchronous reads with at most one writer
- Owner
  - The block is in some other cache
  - We have modified the block, and are the only thread allowed to modify it
  - If we make modifications, it's our responsibility to tell all the other caches in shared state about these changes
  - Allows for writing while other threads read the same data

# Cache Coherence Invariants

For any memory address x, at any given time period (epoch):

Single-Writer, Multiple-Read (SWMR) Invariant

Read-write epoch: there exists only a single processor that may write to x (and can also read it)

Read-Only- epoch: some number of processors that may only read x

Data-Value Invariant (write serialization)

The value of the memory address at the start of an epoch is the same as the value of the memory location at the end of its last read-write epoch

# Question

## WHAT ABOUT SHARED CACHE?

# Shared caches

One single cache shared by all processors

- Eliminates problem of replicating state in multiple caches

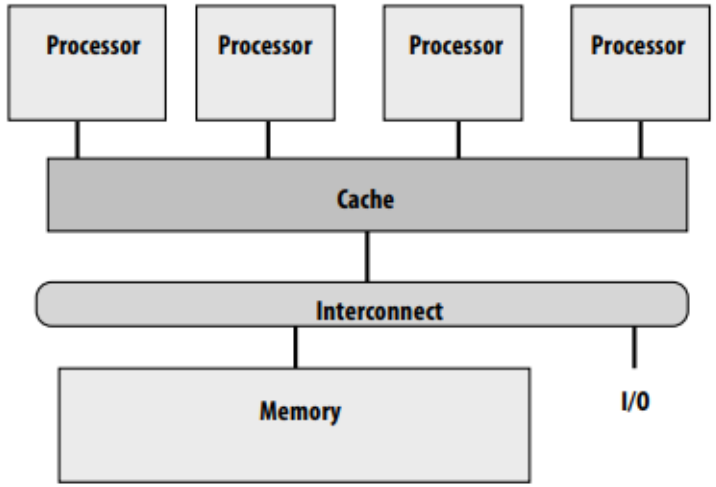
Obvious scalability problems (since the point of a cache is to be local and fast)

- Interference (conflict misses) / contention due to many clients(destructive)

But shared caches can have benefits:

- Facilitates fine-grained sharing (overlapping working sets)
- Loads/stores by one processor might pre-fetch lines for another processor (constructive)

# Shared Cache




```
forall (i= 0; i++; i< N)
   x[i] = y[i] + y[i+1] + y[i+2];
```

# Snooping cache-coherence schemes

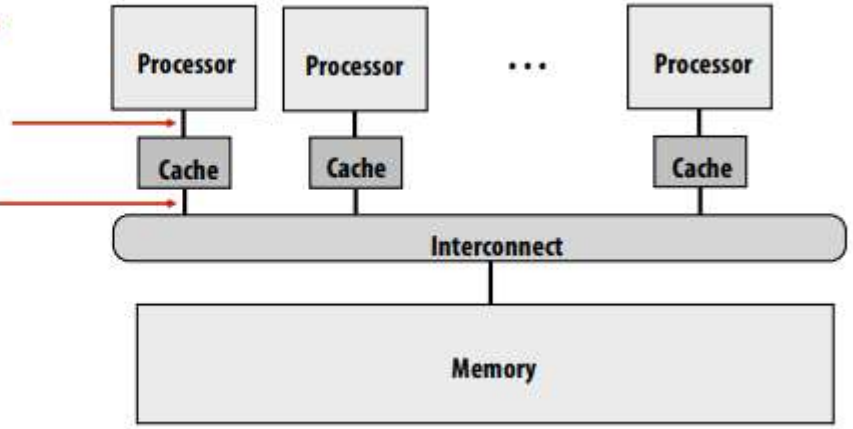
Main idea: all coherence-related activity is broadcast to all processors in the system (more specifically: to the processor's cache controllers)

Cache controllers monitor ("they snoop") memory operations, and follow **cache coherence protocol** to maintain memory coherence

# Snooping cache-coherence

Notice: now cache controller must respond to actions from "both ends":

1. LD/ST requests from its local processor
2. Coherence-related activity broadcast over the chip's interconnect



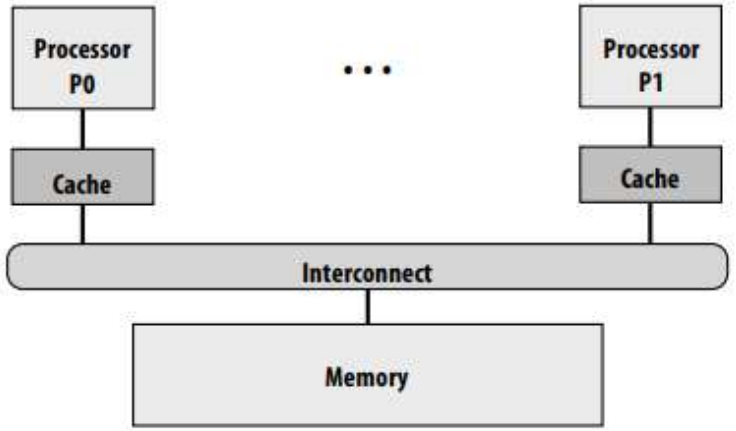
# Simple coherence implementation

**Let's assume:**

1. **Write-through caches**
2. **Granularity of coherence is cache line**

**Coherence Protocol:**

- **Upon write, cache controller broadcasts invalidation message**
- **As a result, the next read from other processors will trigger cache miss**




**(processor retrieves updated value from memory due to write-through policy)**

| Action | Interconnect activity | P0 $ | P1 $ | mem location X |
|---|---|---|---|---|
| | | | | 0 |
| P0 load X | cache miss for X | 0 | | 0 |
| P1 load X | cache miss for X | 0 | 0 | 0 |
| P0 write 100 to X | invalidation for X | 100 | | 100 |
| P1 load X | cache miss for X | 100 | 100 | 100 |

# Question

DO YOU SEE ANY PROBLEM?

(HINT: MAIN MEMORY)

# Write through policy inefficiency

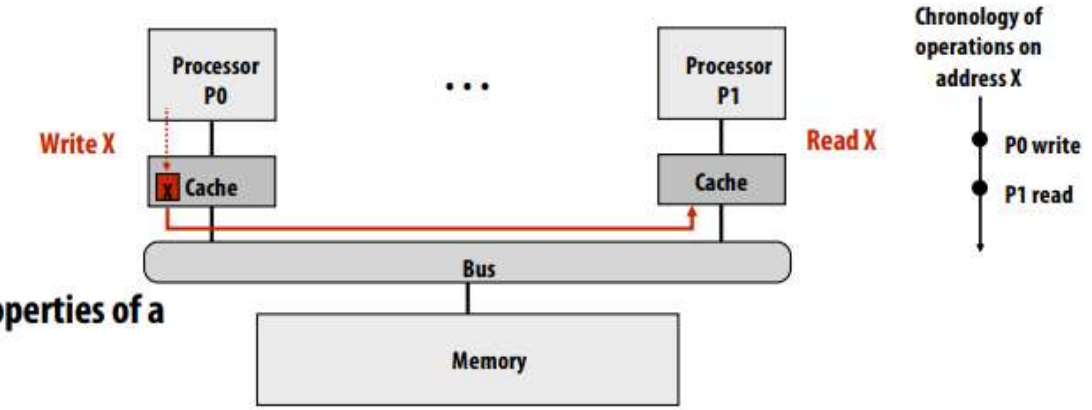
Every write operation goes out to memory

- Very high bandwidth requirements

Write-back caches absorb most write traffic as cache hits

- Significantly reduces bandwidth requirements
- But now how do we maintain cache coherence invariants?
- This requires more sophisticated coherence protocols

# Cache coherence with write back cache




What are two important properties of a bus?

**Dirty state of cache line now indicates exclusive ownership (Read-Write Epoch)**

- Modified: cache is only cache with a valid copy of line (it can safely be written to)
- Owner: cache is responsible for propagating information to other processors when they attempt to load it from memory (otherwise a load from another processor will get stale data from memory)

# Cache Coherence Protocol

Algorithm that maintains cache coherent invariants

The logic we are about to describe is performed by each processor's cache controller in response to:

- Loads and stores by the local processor
- Messages from other caches on the bus

If all cache controllers operate according to this described protocol, then coherence will be maintained

- The caches "cooperate" to ensure coherence is maintained

# Invalidation based write back protocol

Key ideas: A line in the “modified” state can be modified without notifying the other caches

Processor can only write to lines in the modified state

- Need a way to tell other caches that processor wants exclusive access to the line
- We accomplish this by sending message to all the other caches

When cache controller sees a request for modified access to a line it contains

- It must **invalidate** the line in its cache

# MSI write-back invalidation protocol

Key tasks of protocol

- Ensuring processor obtains exclusive access for a write
- Locating most recent copy of cache line's data on cache miss

Three cache line states

- Invalid (I): same as meaning of invalid in uniprocessor cache
- Shared (S): line valid in one or more caches, memory is up to date
- Modified (M): line valid in exactly one cache (a.k.a. "dirty" or "exclusive" state)

# MSI write-back invalidation protocol

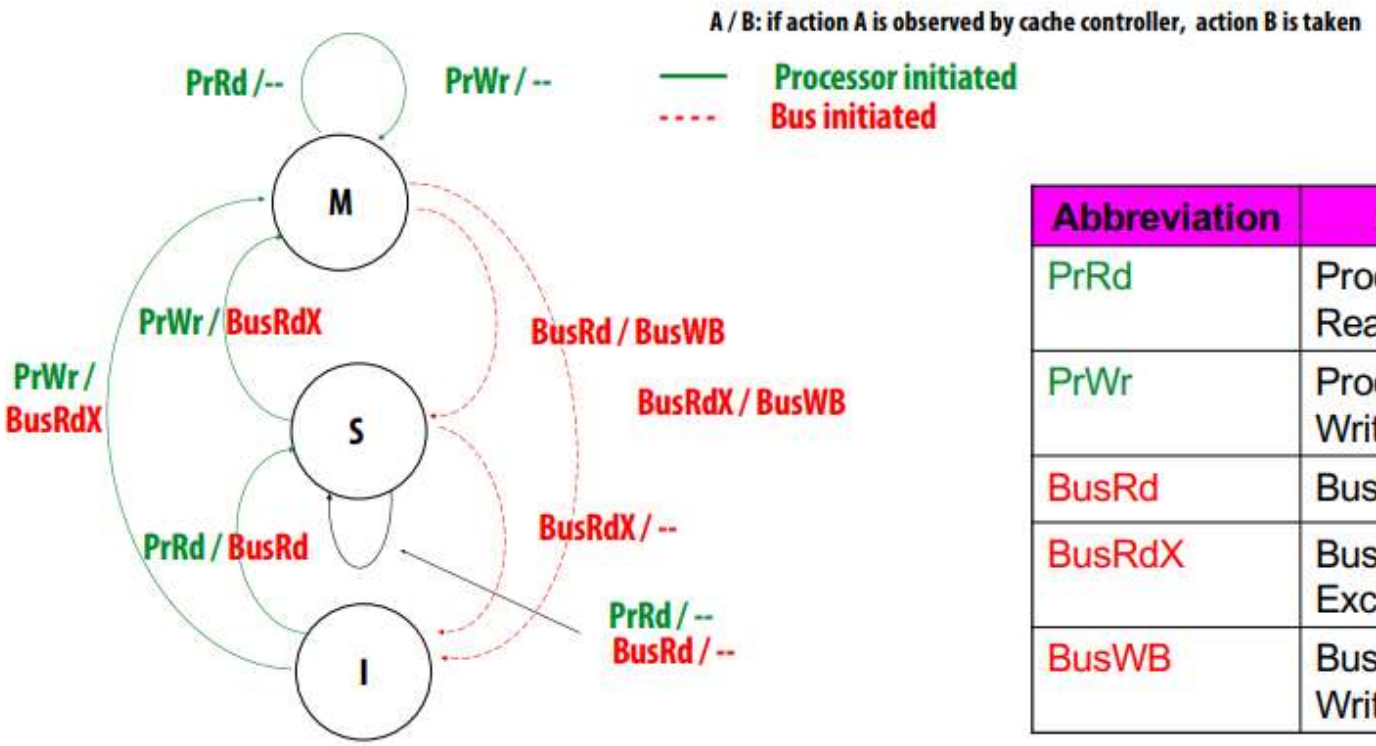
Two processor operations (triggered by local CPU)

- PrRd (read)
- PrWr (write)

Three coherence-related bus transactions (from remote caches)

- BusRd: obtain copy of line with no intent to modify
- BusRdX: obtain copy of line with intent to modify
- BusWB: write dirty line out to memory

# MSI Invalidation Protocol State Diagram




| Abbreviation | Action |
|---|---|
| PrRd | Processor Read |
| PrWr | Processor Write |
| BusRd | Bus Read |
| BusRdX | Bus Read Exclusive |
| BusWB | Bus Writeback |

# MSI Invalidation Protocol

Read obtains block in “shared”

- even if only cached copy

Obtain exclusive ownership before writing

- BusRdX causes others to invalidate
- If M in another cache, will cause writeback
- BusRdX even if hit in S - promote to M (upgrade)

# Example

| Proc Action | P1 $-state | P2 $-state | P3 $-state | Bus Trans | Data from |
|---|---|---|---|---|---|
| P1 read x | S | -- | -- | BusRd | Memory |
| P3 read x | S | -- | S | BusRd | Memory |
| P3 write x | I | -- | M | BusRdX | Memory |
| P1 read x | S | -- | S | BusRd | P3 $ |
| P1 read x | S | -- | S | | P1 $ |
| P2 write x | I | M | I | BusRdX | Memory |
| | | | | | |